U25AI055

Lab-5

PART A: DOUBLY LINKED LIST (ADVANCED)

1. **Reverse a Doubly Linked List Input: 10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ NULL Output: Reversed Doubly Linked List: 40 ⇄ 30 ⇄ 20 ⇄ 10 ⇄ NULL**

```
// A. 1. Reverse a doubly linked list
#include <iostream>
using namespace std;
struct Node {
  Node *previous;
  int data;
  Node *next;
};
void displayList(Node *head);
Node* reverseList(Node *head);
int main() {
  Node *head, *newNode, *temp;
  head = NULL;
  int n;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;
    newNode->next = NULL;

    if (head == NULL) {
      head = newNode;
      temp = newNode;
      temp->previous = NULL;
    } else {
      newNode->previous = temp;
      temp->next = newNode;
      temp = newNode;
    }
  }
  cout << "Elements of list before reversal are ";
  displayList(head);
```

```
    cout << endl;
    head = reverseList(head);
    cout << "Elements of list after reversal are ";
    displayList(head);
    cout << endl;
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* reverseList(Node *head) {
    Node *ptr, *q, *p;
    ptr = head;
    q = p = NULL;
    while (ptr != NULL) {
        p = ptr;
        ptr = ptr->next;
        p->next = q;
        p->previous = ptr;
        q = p;
    }
    head = p;
    return head;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Elements of list before reversal are 10 20      30      40
Elements of list after reversal are 40  30      20      10
```

2. **Find Pair with Given Sum Input: List: 1 ⇄ 2 ⇄ 4 ⇄ 5 ⇄ 6 ⇄ 8 ⇄ 9 ⇄ NULL Sum = 7 Output: Pairs with given sum: (1, 6) (2, 5)**

```
// A. 2. Find pair with given sum
#include <iostream>
```

```
#include <vector>
using namespace std;
struct Node {
   Node *previous;
   int data;
   Node *next;
};
vector<vector<int>> findPair(Node *head, int target);
void displayList(Node *head);
int main() {
   Node *head, *newNode, *temp;
   int n, target;
   head = NULL;
   cout << "Enter number of nodes: ";
   cin >> n;
   for (int i = 0; i < n; i++) {
      newNode = new Node;
      cout << "Enter element " << i + 1 << ": ";
      cin >> newNode->data;
      newNode->next = NULL;

      if (head == NULL) {
         head = newNode;
         temp = newNode;
         temp->previous = NULL;
      } else {
         newNode->previous = temp;
         temp->next = newNode;
         temp = newNode;
      }
   }
   displayList(head);
   cout << endl;
   cout << "Enter target: ";
   cin >> target;
   vector<vector<int>> result = findPair(head, target);
   for (vector<int> pair : result) {
      cout << "(" << pair[0] << ", " << pair[1] << ")\t";
   }
   cout << endl;
   return 0;
```

```
}
void displayList(Node *head) {
   Node *ptr;
   ptr = head;
   while (ptr != NULL) {
      cout << ptr->data << "\t";
      ptr = ptr->next;
   }
}
vector<vector<int>> findPair(Node *head, int target) {
   vector<vector<int>> result;
   Node *p, *q;
   q = head;
   while (q->next != NULL) {
      q = q->next;
   }
   p = head;
   while (p != q && p->next != q) {
      if (p->data + q->data == target) {
         vector<int> pair{p->data, q->data};
         result.push_back(pair);
         p = p->next;
         q = q->previous;
      } else if (p->data + q->data > target) {
         q = q->previous;
      } else {
         p = p->next;
      }
   }
   return result;
}
```

Output:

```
Enter number of nodes: 7
Enter element 1: 1
Enter element 2: 2
Enter element 3: 4
Enter element 4: 5
Enter element 5: 6
Enter element 6: 8
Enter element 7: 9
1       2       4       5       6       8       9
Enter target: 7
(1, 6)  (2, 5)
```

3. **Rotate by k Nodes Input: List: 10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ 50 ⇄ NULL k = 2 Output: List after rotation: 30 ⇄ 40 ⇄ 50 ⇄ 10 ⇄ 20 ⇄ NULL**

```
// A. 3. Rotate by k nodes
#include <iostream>
using namespace std;
struct Node {
  Node *previous;
  int data;
  Node *next;
};
void displayList(Node *head);
Node* rotateList(Node *head, int k);
int main() {
  Node *head, *newNode, *temp;
  int n, k;
  head = NULL;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;
    newNode->next = NULL;
    if (head == NULL) {
      head = newNode;
      temp = newNode;
      temp->previous = NULL;
    } else {
      newNode->previous = temp;
      temp->next = newNode;
      temp = newNode;
```

```
        }
    }
    cout << "Elements of list before rotation are ";
    displayList(head);
    cout << endl;
    cout << "Enter k: ";
    cin >> k;
    head = rotateList(head, k);
    cout << "Elements of list after rotation are ";
    displayList(head);
    cout << endl;
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* rotateList(Node *head, int k) {
    if (head == NULL) {
        return head;
    }
    Node *ptr, *p, *q;
    int i;
    q = head;
    ptr = head;
    for (i = 1; i < k && ptr->next != NULL; i++) {
        ptr = ptr->next;
    }
    if (i == k) {
        p = ptr->next;
        ptr->next = NULL;
        if (p != NULL) {
            p->previous = NULL;
            head = p;
            p = head;
            while (p->next != NULL) {
                p = p->next;
```

```
        }
        p->next = q;
        q->previous = p;
      }
    }
    return head;
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50
Elements of list before rotation are 10 20      30      40      50
Enter k: 2
Elements of list after rotation are 30  40      50      10      20
```

4. **Delete All Duplicate Nodes Input: 5 ⇄ 2 ⇄ 4 ⇄ 2 ⇄ 5 ⇄ 3 ⇄ NULL Output: List after removing all duplicates: 4 ⇄ 3 ⇄ NULL**

```
// A. 4. Delete all duplicate nodes
#include <iostream>
using namespace std;
struct Node {
    Node *previous;
    int data;
    Node *next;
};
void displayList(Node *head);
void removeElement(Node *head, Node *ptr);
Node* deleteAllDuplicates(Node *head);
int main() {
    Node *head, *newNode, *temp;
    int n, k;
    head = NULL;
    cout << "Enter number of nodes: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        newNode = new Node;
        cout << "Enter element " << i + 1 << ": ";
        cin >> newNode->data;
        newNode->next = NULL;
        if (head == NULL) {
```

```
        head = newNode;
        temp = newNode;
        temp->previous = NULL;
      } else {
        newNode->previous = temp;
        temp->next = newNode;
        temp = newNode;
      }
    }
    head = deleteAllDuplicates(head);
    displayList(head);
    cout << endl;
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
      cout << ptr->data << "\t";
      ptr = ptr->next;
    }
}
Node* deleteNode(Node *head, Node *ptr) {
    if (ptr == head) {
      if (ptr->next != NULL) {
        ptr->next->previous = NULL;
      }

      head = ptr->next;

      return head;
    }

    if (ptr->previous != NULL) {
      ptr->previous->next = ptr->next;
    }

    if (ptr->next != NULL) {
      ptr->next->previous = ptr->previous;
    }
    return head;
```

```
}
Node* deleteAllDuplicates(Node *head) {
  Node *p, *q, *ptr, *temp;
  int flag;
  p = head;

  while (p != NULL) {
    q = p->next;
    flag = 0;
    while (q != NULL && flag == 0) {
      if (p->data == q->data) {
        flag = 1;
      }
      q = q->next;
    }

    if (flag == 1) {
      ptr = head;
      while (ptr != NULL) {
        if (ptr->data == p->data) {
          temp = ptr->next;
          head = deleteNode(head, ptr);
          ptr = temp;
        } else {
          ptr = ptr->next;
        }
      }
      p = head;
    } else {
      p = p->next;
    }

  }
  return head;
}
```

Output:

```
Enter number of nodes: 6
Enter element 1: 5
Enter element 2: 2
Enter element 3: 4
Enter element 4: 2
Enter element 5: 5
Enter element 6: 3
4       3
```

5. **Maximum Sum Sublist Input: 2 ⇄ -4 ⇄ 6 ⇄ -3 ⇄ 9 ⇄ -8 ⇄ NULL Output: Maximum sum sublist: 6 ⇄ -3 ⇄ 9 Maximum Sum = 12**

```
// A. 5. Maximum sum sublist
#include <iostream>
using namespace std;
struct Node {
  Node *previous;
  int data;
  Node *next;
};
void displayList(Node *head);
void displayList(Node *beg, Node *end);
int maximumSumSublist(Node *head);
int main() {
  Node *head, *newNode, *temp;
  int n, a, b;
  head = NULL;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;
    newNode->next = NULL;

    if (head == NULL) {
      head = newNode;
      temp = newNode;
      temp->previous = NULL;
    } else {
      newNode->previous = temp;
      temp->next = newNode;
      temp = newNode;
```

```
        }
    }
    displayList(head);
    cout << endl;
    cout << maximumSumSublist(head);
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
void displayList(Node *beg, Node *end) {
    Node *ptr;
    ptr = beg;
    while (ptr != NULL && ptr != end->next) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
int maximumSumSublist(Node *head) {
    Node *ptr, *beg, *end, *temp;
    int result = 0, sum = 0;

    ptr = head;

    beg = head;
    end = head;
    temp = head;

    while (ptr != NULL) {
        sum += ptr->data;

        if (sum > result) {
            result = sum;
            beg = temp;
            end = ptr;
        }
```

```
    if (sum < 0) {
      sum = 0;
      temp = ptr->next;
    }

    ptr = ptr->next;
  }

  displayList(beg, end);
  cout << endl;

  return result;
}
```

Output:

```
Enter number of nodes: 6
Enter element 1: 2
Enter element 2: -4
Enter element 3: 6
Enter element 4: -3
Enter element 5: 9
Enter element 6: -8
2       -4      6       -3      9       -8
6       -3      9
12
```

6. **Palindrome Check Input: 1 ⇄ 2 ⇄ 3 ⇄ 2 ⇄ 1 ⇄ NULL Output: The doubly linked list is a PALINDROME.**

```
// A. 6. Palindrome check
#include <iostream>
using namespace std;
struct Node {
  Node *previous;
  int data;
  Node *next;
};
void displayList(Node *head);
bool checkPalindrome(Node *head);
int main() {
  Node *head, *newNode, *temp;
  int n, k;
  head = NULL;
  cout << "Enter number of nodes: ";
```

```
    cin >> n;
    for (int i = 0; i < n; i++) {
        newNode = new Node;
        cout << "Enter element " << i + 1 << ": ";
        cin >> newNode->data;
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            temp = newNode;
            temp->previous = NULL;
        } else {
            newNode->previous = temp;
            temp->next = newNode;
            temp = newNode;
        }
    }
    displayList(head);
    cout << endl;
    if (checkPalindrome(head)) {
        cout << "Palindrome\n";
    } else {
        cout << "Not palindrome\n";
    }
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
bool checkPalindrome(Node *head) {
    Node *p, *q;
    q = head;
    while (q->next != NULL) {
        q = q->next;
    }
    p = head;
```

```
  while (p->next != q && p != q) {
    if (p->data != q->data) {
      return false;
    }
    p = p->next;
    q = q->previous;
  }
  return true;
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 2
Enter element 5: 1
1       2       3       2       1
Palindrome
```

7. **Merge Two Sorted Doubly Linked Lists Input: List1: 1 ⇄ 3 ⇄ 5 ⇄ NULL List2: 2 ⇄ 4 ⇄ 6 ⇄ NULL Output: Merged Sorted Doubly Linked List: 1 ⇄ 2 ⇄ 3 ⇄ 4 ⇄ 5 ⇄ 6 ⇄ NULL**

```
// 7. Merge two sorted doubly linked lists
#include <iostream>
using namespace std;
struct Node {
  Node *previous;
  int data;
  Node *next;
};
void displayList(Node *head);
Node* mergeLists(Node *head1, Node *head2);
int main() {
  Node *head, *head1, *head2, *newNode, *temp;
  int n;
  head1 = NULL;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;
```

```
        newNode->next = NULL;
        if (head1 == NULL) {
            head1 = newNode;
            temp = newNode;
            temp->previous = NULL;
        } else {
            newNode->previous = temp;
            temp->next = newNode;
            temp = newNode;
        }
    }
    head2 = NULL;
    cout << "Enter number of nodes: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        newNode = new Node;
        cout << "Enter element " << i + 1 << ": ";
        cin >> newNode->data;
        newNode->next = NULL;
        if (head2 == NULL) {
            head2 = newNode;
            temp = newNode;
            temp->previous = NULL;
        } else {
            newNode->previous = temp;
            temp->next = newNode;
            temp = newNode;
        }
    }
    displayList(head1);
    cout << endl;
    displayList(head2);
    cout << endl;
    head = mergeLists(head1, head2);
    displayList(head);
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
```

```
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* mergeLists(Node *head1, Node *head2) {
    if (head1 == NULL) {
        return head2;
    } else if (head2 == NULL) {
        return head1;
    }
    Node *head, *ptr, *p, *q;
    p = head1;
    q = head2;
    if (p->data <= q->data) {
        ptr = p;
        p = p->next;
    } else {
        ptr = q;
        q = q->next;
    }
    head = ptr;
    while (p != NULL && q != NULL) {
        if (p->data <= q->data) {
            ptr->next = p;
            p->previous = ptr;
            p = p->next;
        } else {
            ptr->next = q;
            q->previous = ptr;
            q = q->next;
        }
        ptr = ptr->next;
    }
    if (p != NULL) {
        ptr->next = p;
        p->previous = ptr;
    } else if (q != NULL) {
        ptr->next = q;
        q->previous = ptr;
    }
    return head;
```

}

Output:

```
Enter number of nodes: 3
Enter element 1: 1
Enter element 2: 3
Enter element 3: 5
Enter number of nodes: 3
Enter element 1: 2
Enter element 2: 4
Enter element 3: 6
1       3       5
2       4       6
1       2       3       4       5       6
```

8. **Swap Nodes at Given Positions Input: List: 10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ NULL Swap positions 2 and 4 Output: List after swapping nodes: 10 ⇄ 40 ⇄ 30 ⇄ 20 ⇄ NULL**

```
// A. 8. Swap nodes at given positions
#include <iostream>
using namespace std;
struct Node {
  Node *previous;
  int data;
  Node *next;
};
void displayList(Node *head);
Node* swapNodes(Node *head, int a, int b);
int main() {
  Node *head, *newNode, *temp;
  int n, a, b;
  head = NULL;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;
    newNode->next = NULL;

    if (head == NULL) {
      head = newNode;
      temp = newNode;
      temp->previous = NULL;
```

```
        } else {
            newNode->previous = temp;
            temp->next = newNode;
            temp = newNode;
        }
    }
    displayList(head);
    cout << endl;
    cout << "Enter positions: ";
    cin >> a >> b;
    head = swapNodes(head, a, b);
    displayList(head);
    cout << endl;
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* swapNodes(Node *head, int a, int b) {
    Node *p, *q, *pPrevious, *qPrevious, *pNext, *qNext;
    p = head;
    for (int i = 1; i < a && p != NULL; i++) {
        p = p->next;
    }
    q = head;
    for (int i = 1; i < b && q != NULL; i++) {
        q = q->next;
    }

    pPrevious = p->previous;
    pNext = p->next;
    qPrevious = q->previous;
    qNext = q->next;

    if (p->next == q) {
        if (p == head) {
```

```
        head = q;
    }
    if (p->previous != NULL) {
        p->previous->next = q;
    }

    if (q->next != NULL) {
        q->next->previous = p;
    }

    q->previous = pPrevious;
    q->next = p;

    p->previous = q;
    p->next = qNext;
} else {
    if (p == head) {
        head = q;
    }

    if (p->previous != NULL) {
        p->previous->next = q;
    }

    if (p->next != NULL) {
        p->next->previous = q;
    }

    if (q->previous != NULL) {
        q->previous->next = p;
    }

    if (q->next != NULL) {
        q->next->previous = p;
    }

    p->previous = qPrevious;
    p->next = qNext;

    q->previous = pPrevious;
    q->next = pNext;
```

```
    }
    return head;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
10      20      30      40
Enter positions: 2
4
10      40      30      20
```

9. **Clone a Doubly Linked List with Random Pointer Output: Original and cloned lists are identical. Random pointers copied successfully**

```
// A. 9. Clone doubly linked list with random pointers
#include <iostream>
using namespace std;
struct Node {
    Node *previous;
    int data;
    Node *next;
    Node *random;
};
void displayList(Node *head);
Node* cloneList(Node *head);
int main() {
    Node *head, *node1, *node2, *node3, *node4;
    node1 = new Node;
    node2 = new Node;
    node3 = new Node;
    node4 = new Node;

    head = node1;

    node1->data = 1;
    node1->next = node2;
    node1->previous = NULL;
    node1->random = node3;

    node2->data = 3;
```

```
    node2->next = node3;
    node2->previous = node1;
    node2->random = node3;

    node3->data = 5;
    node3->next = node4;
    node3->previous = node2;
    node3->random = NULL;

    node4->data = 9;
    node4->next = NULL;
    node4->previous = node3;
    node4->random = node3;

    Node *clone = cloneList(head);
    cout << "List cloned";
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* cloneList(Node *head) {
    Node *clone, *newNode, *temp, *originalNext, *duplicateNext, *duplicate,
*original;

    clone = NULL;
    original = head;

    while (original != NULL) {
        newNode = new Node;
        newNode->data = original->data;
        newNode->next = NULL;
        newNode->previous = NULL;
        if (clone == NULL) {
            clone = newNode;
            temp = newNode;
```

```
    } else {
        newNode->previous = temp;
        temp->next = newNode;
        temp = newNode;
    }
    original = original->next;
}

duplicate = clone;
original = head;

while (duplicate != NULL)
{
    originalNext = original->next;

    original->next = duplicate;
    duplicate->previous = original;

    duplicateNext = duplicate->next;

    duplicate->next = originalNext;

    if (originalNext != NULL) {
        originalNext->previous = duplicate;
    }

    original = originalNext;
    duplicate = duplicateNext;
}

original = head;

while (original != NULL && original->next != NULL) {
    if (original->random != NULL) {
        original->next->random = original->random->next;
    }
    original = original->next->next;
}

original = head;
duplicate = head->next;
```

```
    while (original != NULL) {
        originalNext = original->next;
        original->next = originalNext->next;

        if (original->next != NULL) {
            original->next->previous = original;
        }

        if (duplicate->next != NULL) {
            duplicate->next = duplicate->next->next;

            if (duplicate->next != NULL) {
                duplicate->next->previous = duplicate;
            }
        }

        original = original->next;
        duplicate = duplicate->next;
    }
    return clone;
}
```

Output:

```
List cloned
```

PART B: CIRCULAR LINKED LIST (ADVANCED)

1. **Split Circular Linked List Input: 10 → 20 → 30 → 40 → 50 → 60 → (back to 10) Output: First Half: 10 → 20 → 30 → (back to 10) Second Half: 40 → 50 → 60 → (back to 40)**

```
// B. 1. Split circular linked list
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node *next;
};
void displayList(Node *head);
void splitList(Node *head);
int main() {
    int n;
```

```
    Node *head, *newNode, *temp;

    head = NULL;

    cout << "Enter number of nodes: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        newNode = new Node;

        cout << "Enter element " << i + 1 << ": ";
        cin >> newNode->data;

        newNode->next = head;

        if (head == NULL) {
            head = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }

    displayList(head);
    cout << endl;

    splitList(head);
    return 0;
}
void displayList(Node *head) {
    Node *ptr;

    ptr = head;

    do
    {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    } while (ptr != head);
}
```

```
void splitList(Node *head) {
   Node *slow, *fast, *another, *ptr;

   slow = head;
   fast = head;

   do {
      slow = slow->next;
      fast = fast->next->next;
   } while (fast != head && fast->next != head);

   ptr = head;

   while (ptr->next != slow) {
      ptr = ptr->next;
   }

   ptr->next = head;

   another = slow;

   ptr = another;

   while (ptr->next != head) {
      ptr = ptr->next;
   }

   ptr->next = another;

   displayList(head);
   cout << endl;
   displayList(another);
   cout << endl;
}
```

Output:

```
Enter number of nodes: 6
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50
Enter element 6: 60
10      20      30      40      50      60
10      20      30
40      50      60
```

2. **Check if Circular List is Sorted Input: 1 → 3 → 5 → 7 → (back to 1) Output: The circular linked list is SORTED.**

```
// B. 2. Check is circular list is sorted
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
};
void displayList(Node *head);
bool checkSortedList(Node *head);
int main() {
  int n;
  Node *head, *newNode, *temp;

  head = NULL;

  cout << "Enter number of nodes: ";
  cin >> n;

  for (int i = 0; i < n; i++) {
    newNode = new Node;

    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;

    newNode->next = head;

    if (head == NULL) {
      head = newNode;
      temp = newNode;
    } else {
      temp->next = newNode;
```

```
        temp = newNode;
      }
    }

    displayList(head);
    cout << endl;

    if (checkSortedList(head)) {
      cout << "Sorted in ascending order\n";
    } else {
      cout << "Not sorted in ascending order\n";
    }
    return 0;
}
void displayList(Node *head) {
    Node *ptr;

    ptr = head;

    do
    {
      cout << ptr->data << "\t";
      ptr = ptr->next;
    } while (ptr != head);
}
bool checkSortedList(Node *head) {
    Node *ptr;
    ptr = head;
    do {
      if (ptr->data > ptr->next->data) {
        return false;
      }
      ptr = ptr->next;
    } while (ptr->next != head);
    return true;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 1
Enter element 2: 3
Enter element 3: 5
Enter element 4: 7
1       3       5       7
Sorted in ascending order
```

3. **Insert into Sorted Circular Linked List Input: Insert 6 into: 1 → 3 → 5 → 7 → (back to 1) Output: After insertion: 1 → 3 → 5 → 6 → 7 → (back to 1)**

```
// B. 3. Insert into sorted circular linked list
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
};
void displayList(Node *head);
Node *insertIntoSortedList(Node *head, int item);
int main() {
  int n, item;
  Node *head, *newNode, *temp;

  head = NULL;

  cout << "Enter number of nodes: ";
  cin >> n;

  for (int i = 0; i < n; i++) {
    newNode = new Node;

    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;

    newNode->next = head;

    if (head == NULL) {
      head = newNode;
      temp = newNode;
    } else {
      temp->next = newNode;
      temp = newNode;
```

```
        }
    }

    displayList(head);
    cout << endl;

    cout << "Enter item: ";
    cin >> item;

    head = insertIntoSortedList(head, item);

    displayList(head);
    return 0;
}
void displayList(Node *head) {
    Node *ptr;

    ptr = head;

    do
    {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    } while (ptr != head);
}
Node *insertIntoSortedList(Node *head, int item) {
    Node *ptr, *newNode;
    newNode = new Node;
    newNode->data = item;

    if (head->data > item) {
        newNode->next = head;



        ptr = head;

        while (ptr->next != head) {
            ptr = ptr->next;
        }
        ptr->next = newNode;
```

```
    head = newNode;
    return head;
  }

  ptr = head;

  do {
    if (ptr->next->data > item) {

      newNode->next = ptr->next;
      ptr->next = newNode;

      break;
    }
    ptr = ptr->next;
  } while (ptr->next != head);

  if (ptr->next == head) {
    newNode->next = head;
    ptr->next = newNode;
  }
  return head;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 1
Enter element 2: 3
Enter element 3: 5
Enter element 4: 7
1       3       5       7
Enter item: 6
1       3       5       6       7
```

4. **Reverse Circular Linked List Input: 1 → 2 → 3 → 4 → (back to 1) Output: Reversed Circular Linked List: 4 → 3 → 2 → 1 → (back to 4)**

```
// B. 5. Reverse circular linked list
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
```

```
};
void displayList(Node *head);
Node* reverseList(Node *head);
int main() {
   int n;
   Node *head, *newNode, *temp;

   head = NULL;

   cout << "Enter number of nodes: ";
   cin >> n;

   for (int i = 0; i < n; i++) {
      newNode = new Node;

      cout << "Enter element " << i + 1 << ": ";
      cin >> newNode->data;

      newNode->next = head;

      if (head == NULL) {
         head = newNode;
         temp = newNode;
      } else {
         temp->next = newNode;
         temp = newNode;
      }
   }

   displayList(head);
   cout << endl;

   head = reverseList(head);

   displayList(head);
   cout << endl;
   return 0;
}
void displayList(Node *head) {
   Node *ptr;
```

```
    ptr = head;

    do
    {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    } while (ptr != head);
}
Node* reverseList(Node *head) {
    Node *ptr, *slow, *fast;

    ptr = head;
    while (ptr->next != head) {
        ptr = ptr->next;
    }
    ptr->next = NULL;

    ptr = head;
    slow = NULL;
    fast = NULL;

    while (ptr != NULL) {
        fast = ptr;
        ptr = ptr->next;
        fast->next = slow;
        slow = fast;
    }

    ptr = head;

    while (ptr->next != NULL) {
        ptr = ptr->next;
    }

    ptr->next = fast;

    head = fast;

    return head;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
1       2       3       4
4       3       2       1
```

5. **Delete Every k-th Node Input: List: 1 → 2 → 3 → 4 → 5 → (back to 1) k = 2 Output: Remaining node: 3**

```
// B. 5. Delete every kth node
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
};
void displayList(Node *head);
Node *deleteEveryKthNode(Node *head, int k);
int main() {
  int n;
  Node *head, *newNode, *temp;

  head = NULL;

  cout << "Enter number of nodes: ";
  cin >> n;

  for (int i = 0; i < n; i++) {
    newNode = new Node;

    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;

    newNode->next = head;

    if (head == NULL) {
      head = newNode;
      temp = newNode;
    } else {
      temp->next = newNode;
```

```
        temp = newNode;
      }
    }

    displayList(head);
    cout << endl;

    head = deleteEveryKthNode(head, 2);

    displayList(head);
    cout << endl;

    return 0;
}
void displayList(Node *head) {
    Node *ptr;

    ptr = head;

    do
    {
      cout << ptr->data << "\t";
      ptr = ptr->next;
    } while (ptr != head);
}
Node* deleteNode(Node *head, Node *ptr) {
    Node *p;
    if (ptr == head) {
      p = head;
      while (p->next != head) {
        p = p->next;
      }
      p->next = ptr->next;
      head = ptr->next;
      delete ptr;
      return head;
    }

    p = head;
    while (p->next != ptr) {
      p = p->next;
```

```
    }
    p->next = ptr->next;
    delete ptr;
    return head;
}
Node *deleteEveryKthNode(Node *head, int k) {
    Node *ptr, *temp;
    ptr = head;
    while (ptr->next != ptr) {
        for (int i = 1; i < k; i++) {
            ptr = ptr->next;
        }
        temp = ptr->next;
        head = deleteNode(head, ptr);
        ptr = temp;

        displayList(head);
        cout << endl;
    }
    return head;
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
1       2       3       4       5
1       3       4       5
1       3       5
3       5
3
3
```